Software Reliability Coursework 1
Implement a SAT fuzzer

Cristian Cadar (lecturer)
Bachir Bendrissou and Max Stupple (course assistants)

To be undertaken in groups of 3 (or pairs if necessary, with approval)

1 Introduction

In this coursework, you will implement a DIMACS fuzzer. DIMACS is a format describing a boolean formula in conjunctive normal form (CNF). Some information about this format can be found at the following links:

- `https://fairmut3x.wordpress.com/2011/07/29/cnf-conjunctive-normal-form-dimacs-format-explained/`
- `https://srg.doc.ic.ac.uk/70024/papers/satformat.pdf`

The aim of the fuzzer is to find bugs in SAT solvers that were built in C by some of your peers in previous years. Your fuzzer will take as input one of these SAT solvers. We refer to the SAT solver that your fuzzer is applied to as SUT, which stands for *system under test*.

Each group will be provided with a GitLab repository containing the provided files and some sample inputs. This repository should be used for development as you will submit your code using the departmental LabTS service. More details can be found in Section 7.

2 Fuzzer interface

Your fuzzer should be invoked as follows:

```
fuzz-sat /path/to/SUT /path/to/inputs seed
```

where:

- `fuzz-sat` is your fuzzer;
- `/path/to/SUT` refers to the source directory of the SUT;
- `/path/to/inputs` refers to a directory containing a non-empty set of well-formed DIMACS-format files;
- `seed` is an integer that you should use to initialize a random number generator if you wish.

You may use any standard programming language to implement your fuzzer,[1] along with any standard libraries, as long as these libraries do not themselves provide fuzzing capabilities. You should provide a **build.sh** executable bash script that builds your fuzzer, using tools available on a standard lab machine with network access. After building your fuzzer, it should be possible for us to run the fuzzer using a command line of the form shown above.

Your fuzzer is expected to run on the lab machines. Ensure that you thoroughly test the build infrastructure you provide to us, to ensure that your fuzzer builds correctly and that no dependencies are missing. To do this, you can use the tests available on LabTS, which are further explained in Section 7.

The coursework is open-ended, and advice on how to manage your time is given in Section 6. The course reading list suggests a number of papers and sources on fuzzing that might give you inspiration for the techniques that you implement.

[1] Please ask if you are unsure whether your language of choice is sufficiently well-known; our only concern here relates to the teaching team's ability to understand your code when marking.

3 SUT description

3.1 SUT source code layout

We strongly encourage you to implement some form of greybox fuzzing. Therefore the SUT needs to be instrumented to gather feedback information like coverage. To simplify this process, you can assume the SUT is written in C, and generated binaries are already instrumented for you. It lives in a folder containing the following files:

- `sat.c`: the main file for the SAT solver.
- `*.c, *.h`: any number of source and header files in the root of the directory needed for the solver.
- `Makefile`: used to compile the SUT via the `make` command. The makefile has a single target: `ub`. This target compiles the SUT in a manner such that coverage instrumentation using gcov has been added, and support for instrumentation with ASan (without leak detection) and UBSan has been enabled.[2]
- `runsat.sh`: used to run the SUT using the following invocation `/path/to/SUT/runsat.sh /path/to/formula.cnf`.

None of those files should be modified.

3.2 Expected input and output

The SUT is expected to take exactly one argument as input. The argument should be a path to a file containing a formula in CNF DIMACS format. The SUT is expected to check the given formula's satisfiability and print the result to standard output. In case the formula is satisfiable, it should also print a model (that is, a solution) for the formula. Note that there may be multiple models that can satisfy a given formula. If that is the case, the SUT may print any such model. Furthermore, it is acceptable for multiple SUT runs on the same input formula to lead to different models being printed.

A model is represented by a list of integers, corresponding to the variables in the logical formula. A positive integer means that the corresponding variable should be *True*, while a negative integer indicates a *False* value.

3.2.1 Example

Consider the file `formula.cnf`, located within the provided `inputs` directory, containing the following:

```
p cnf 3 2
1 2 0
-1 3 0
```

This corresponds to the logical formula:

$$(A \vee B) \wedge (\neg A \vee C)$$

The formula is satisfiable, and a valid model is:

$$A = \bot; B = C = \top$$

In this case, if the SAT solver succeeds in deducing that the formula is satisfiable and computes this particular model, it should produce (each printed line should end in a new line):

```
$ ./runsat.sh formula.cnf
SAT
-1 2 3
```

The model is not unique, and the SUT may produce any of the following models:

- 1 2 3
- 1 -2 3
- -1 2 -3
- -1 2 3

[2]For simplicity, we do not consider MemorySanitizer and LeakSanitizer during this coursework.

4 Fuzzer Requirements

Your fuzzer should assume that the SUT has been built with the provided makefile using the `ub` target, which enables ASan, UBSan, and gathering of coverage information.

In order to properly log coverage information in the presence of sanitizers, the sanitizers need to be instructed not to abort execution once they identify potential bugs. We provide a wrapper script, `runsat.sh`, that sets environment variables appropriately and invokes the SUT in a manner that ensures that gcov files are correctly generated. This wrapper script will already be in place in the root of each SUT which we provide. To run the SUT, you should thus invoke this script, specifying as a command-line argument the DIMACS file to be given to the SUT. **You should not modify the `runsat.sh` script.**

Your fuzzer should create a folder in the current directory, `fuzzed-tests`, in which to save up to 20 test cases of interest. Your fuzzer should then operate in an infinite loop as follows:

1. Produce an input
2. Run the SUT (via `runsat.sh`) on the input, killing the SUT if a timeout of your choice is reached
3. If execution triggers an undefined behaviour, consider saving the input in `fuzzed-tests`
4. Go to step 1

It is up to you how you handle Step 1, producing an input. You could take a generation-based approach, making inputs from scratch, or a mutation-based approach, modifying existing or previously-generated inputs. You could employ dumb fuzzing, generating or mutating without reference to the DIMACS format, or a form of smart fuzzing, generating DIMACS or DIMACS-like inputs, or mutating existing inputs in a manner that partially or wholly respects the DIMACS format. It is also up to you whether you try to maximise coverage in a blind fashion, by simply generating a diverse range of inputs that are likely to give high coverage, or whether you do so in a feedback-directed fashion, using coverage information to guide input production. For the latter, it is up to you to learn how to use coverage information provided by gcov.

Step 2 is straightforward, but you should decide on an appropriate timeout for which to run the SUT. A timeout is important because (a) the SUT may get stuck in an infinite loop, and (b) SAT solving can be expensive. The challenge here is to find a timeout small enough to achieve a high fuzzing rate, yet large enough to ensure that interesting parts of the SUT are explored. As detailed in Section 5 the evaluation of your fuzzers measures total SUT code coverage achieved during fuzzing. However, killing the SUT with `SIGKILL` (9) prevents the coverage information for the SUT from being generated. Instead, you should kill the SUT using `SIGTERM` (15) or `SIGINT` (2). The SUTs have been modified to generate coverage information before exiting when killed using these signals.

Step 3 involves determining whether the input has triggered an undefined behaviour, which you should do by inspecting any output produced by the sanitizer, and saving the input in the `fuzzed-tests` directory if it looks interesting. Initially you should fill up this directory with the first 20 inputs your fuzzer finds that trigger undefined behaviours. When the directory contains 20 tests, you must have an eviction policy in place that determines whether to replace an existing test with a new test. Your aim here should be to maximise diversity, first aiming to showcase as many different types of undefined behaviour as possible. Second, try to find undefined behaviours triggered by as many distinct source locations as possible.

You are free to employ parallelism inside your fuzzer. We will evaluate your fuzzer on a machine with at least four cores.

5 Deadlines and marking

Groups The deadline for submitting your group information is Friday, 10 October 2025 at 14:00. Group formation for the coursework will be through Scientia. In your groups, choose one of you to be the "group leader". That person should then open the group formation exercise on Scientia and use the drop down menu to add the other group members. Those should then go to Scientia and accept the invitation. If you cannot find partners and you wish to take the course, first post an announcement on Ed Discussion stating that you are looking for a group. If this fails, please email the course staff before the group formation deadline.

Submission The deadline for submitting the coursework itself is Friday, 24 October 2025 at 19:00. The code will be submitted via LabTS, while the report via Scientia (see Section 7 for details).

Marking The final mark will be comprised of an automated testing phase (65% of the marks) and a report detailing your fuzzing strategies (35% of the marks). When we evaluate your fuzzer, we will run it for 30 minutes-per-SUT on a range of SUTs. For each SUT, we will score you based on:

- The cumulative coverage achieved **during fuzzing**. You do not need to keep track of this, as our testing infrastructure will compute it; however, please ensure that your fuzzer does not modify or delete the files produced by gcov.
- The diversity of bug-inducing tests that you save in the `fuzzed-tests` directory. If there are more than 20 tests in this directory, we will evaluate your fuzzer based on 20 tests selected at random.

6 Managing your workload

This coursework project is deliberately open-ended. It is up to you how far you wish to implement interesting fuzzing strategies, intricate formula transformations, etc. We hope that many of you will have fun implementing adventurous fuzzers in the time that you have. Do bear in mind that there is no such thing as a perfect fuzzer. As with any open-ended coursework, at some point, you hit diminishing returns with respect to time invested and marks achieved.

At some point you need to make a call as to when enough is enough, and submit!

7 Submission

7.1 Report

You should submit a PDF file called **report.pdf** via Scientia describing your approach to fuzzing. The report should be a maximum of 4 pages, using 11pt Arial font and 2cm margins. It should present the design choices you have made; details of any optimizations you may have implemented; the way you have evaluated your fuzzer; and design ideas you experimented with that did not make it into the final tool.

7.2 Code

After the group formation deadline (Friday, 10 October 2025 at 14:00), each group will be provided with a skeleton repository containing the provided files, some sample SUTs, and some sample inputs (the purpose of everything is documented locally through some `README.md` files). You will be able to run some tests on your fuzzers through the departmental LabTS system (`https://teaching.doc.ic.ac.uk/labts`), but you are encouraged to do more testing. The tests run on LabTS just check that the fuzzer's interface and outputs are as expected, they do not assess the quality of your fuzzer. The skeleton source code can also be found in Scientia, although the group must work in the assigned repository once created.

To use LabTS, you should log in using your College credentials. Once you do so, you should reach a landing page similar to what can be seen in Figure 1. To access this coursework, you should click on the line labeled "Fuzzing" as seen in the red box.

Every commit you make to the group repository will appear as a separate line in a table, as shown in Figure 2. You are allowed to run the output checks once per-commit; to do so, you should click the "Request test" button on the right-most column of the commit line.

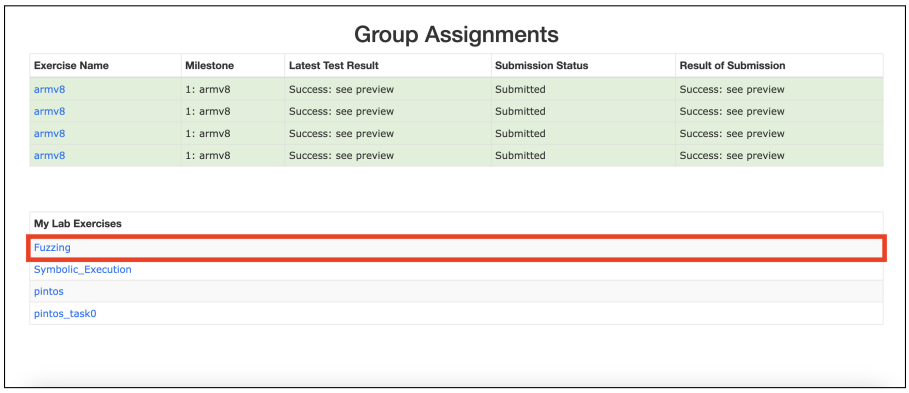
Once the tests have run, the results are available in the "Test Result" column. Furthermore, you can see the test log by clicking "results" in the "Testing Status" column. You can also view a PDF with full details of your submission and test results by clicking "preview" or "final files" in the same column. This may be quite large, as it contains the full code submitted, but notably it also summarises the test results (first page) and contains any error messages (last page). This may be useful if it is hard to find the error message in the test log.

When you are happy with your fuzzer, you can submit the source code by clicking the "Submit" button to the left of the commit you wish to be assessed. Note that you may submit as many different commits as you wish prior to the deadline, but only the last one submitted, which will be highlighted in green, will be marked. If the submit button has turned red and reads "Submit this commit late", this means you have passed the deadline for submitting the coursework, so any commit submitted will be marked as a late submission (capped at 50%).

7.3 GenAI policy

To get the most out of this coursework, we recommend not to use GenAI to complete the assignment. This being said, the use of GenAI tools is permitted, as long as they are not used to complete the entire assignment, but rather help with some specific aspects.

Your report should have a section called "GenAI use" which describes how GenAI was used in your solution. If it has not been used, just add a sentence stating this.



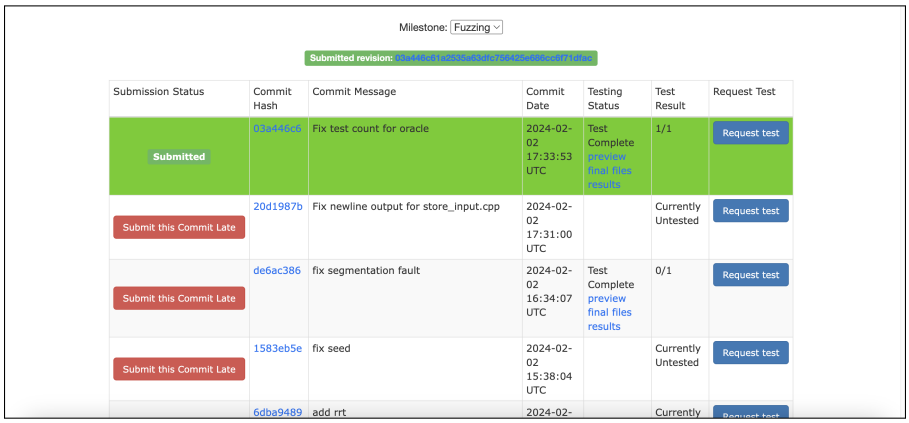
Group Assignments

Exercise Name	Milestone	Latest Test Result	Submission Status	Result of Submission
armv8	1: armv8	Success: see preview	Submitted	Success: see see preview
armv8	1: armv8	Success: see preview	Submitted	Success: see see preview
armv8	1: armv8	Success: see preview	Submitted	Success: see see preview
armv8	1: armv8	Success: see preview	Submitted	Success: see see preview

My Lab Exercises

- Fuzzing
- Symbolic_Execution
- pintos
- pintos_task0

Figure 1: LabTS Home Page

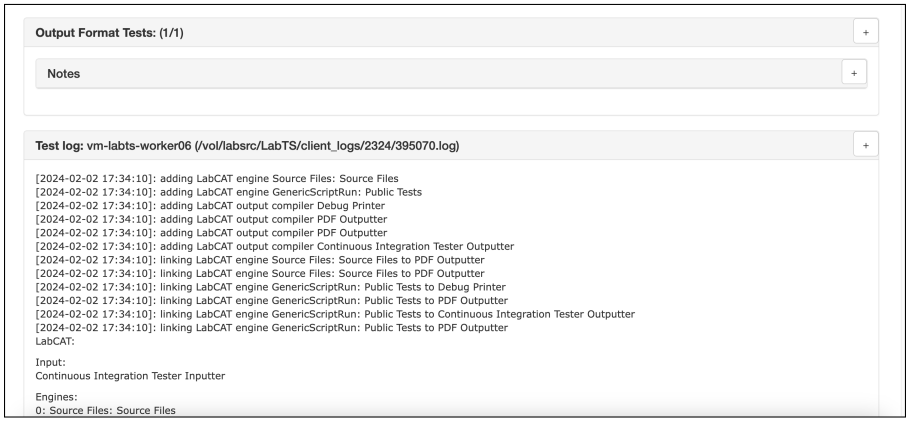


Milestone: Fuzzing

Submitted revision: 03a446c61a2535a63dfc706425e608cc6f71dfac

Submission Status	Commit Hash	Commit Message	Commit Date	Testing Status	Test Result	Request Test
Submitted	03a446c6	Fix test count for oracle	2024-02-02 17:33:53 UTC	Test Complete preview final files results	1/1	Request test
Submit this Commit Late	20d1987b	Fix newline output for store_input.cpp	2024-02-02 17:31:00 UTC		Currently Untested	Request test
Submit this Commit Late	de6ac386	fix segmentation fault	2024-02-02 16:34:07 UTC	Test Complete preview final files results	0/1	Request test
Submit this Commit Late	1583eb5e	fix seed	2024-02-02 15:38:04 UTC		Currently Untested	Request test
	6dba9489	add rrt	2024-02-		Currently	Request test

Figure 2: LabTS Commit List



Output Format Tests: (1/1) +

Notes +

Test log: vm-labts-worker06 (/vol/labsrc/LabTS/client_logs/2324/395070.log) +

```
[2024-02-02 17:34:10]: adding LabCAT engine Source Files: Source Files
[2024-02-02 17:34:10]: adding LabCAT engine GenericScriptRun: Public Tests
[2024-02-02 17:34:10]: adding LabCAT output compiler Debug Printer
[2024-02-02 17:34:10]: adding LabCAT output compiler PDF Outputter
[2024-02-02 17:34:10]: adding LabCAT output compiler PDF Outputter
[2024-02-02 17:34:10]: adding LabCAT output compiler Continuous Integration Tester Outputter
[2024-02-02 17:34:10]: linking LabCAT engine Source Files: Source Files to PDF Outputter
[2024-02-02 17:34:10]: linking LabCAT engine Source Files: Source Files to PDF Outputter
[2024-02-02 17:34:10]: linking LabCAT engine GenericScriptRun: Public Tests to Debug Printer
[2024-02-02 17:34:10]: linking LabCAT engine GenericScriptRun: Public Tests to PDF Outputter
[2024-02-02 17:34:10]: linking LabCAT engine GenericScriptRun: Public Tests to Continuous Integration Tester Outputter
[2024-02-02 17:34:10]: linking LabCAT engine GenericScriptRun: Public Tests to PDF Outputter
LabCAT:

Input:
Continuous Integration Tester Inputter

Engines:
0: Source Files: Source Files
```

Figure 3: LabTS Test Results